

IF3140 - Sistem Basis Data

Praktikum 0

Review Basis Data Relasional dengan PostgreSQL

14 September 2026



Raymond Jonathan Dwi Putra Julianto

13524059

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Petunjuk

1. Praktikum ini bertujuan untuk membiasakan Anda dengan lingkungan DBMS PostgreSQL. PostgreSQL akan menjadi DBMS utama dalam rangkaian pembelajaran IF3140 Sistem Basis Data.
2. Setiap peserta kuliah **WAJIB** mengunduh, menginstal, menjalankan, dan mengerjakan praktikum ini pada server PostgreSQL pada perangkat masing masing.
3. Gunakan soal ini sebagai pembiasaan Anda dalam menggunakan PostgreSQL. Risiko tidak mengerjakan praktikum ini ditanggung oleh masing-masing. Soal praktikum akan memiliki tingkat kesulitan yang setara. Pengerjaan dengan batas waktu dapat menjadi opsi untuk berlatih.
4. Seluruh *keyword* yang ada diperbolehkan pada praktikum ini. Silahkan memilih *keyword* yang sesuai dengan kebutuhan soal.

Bioskop

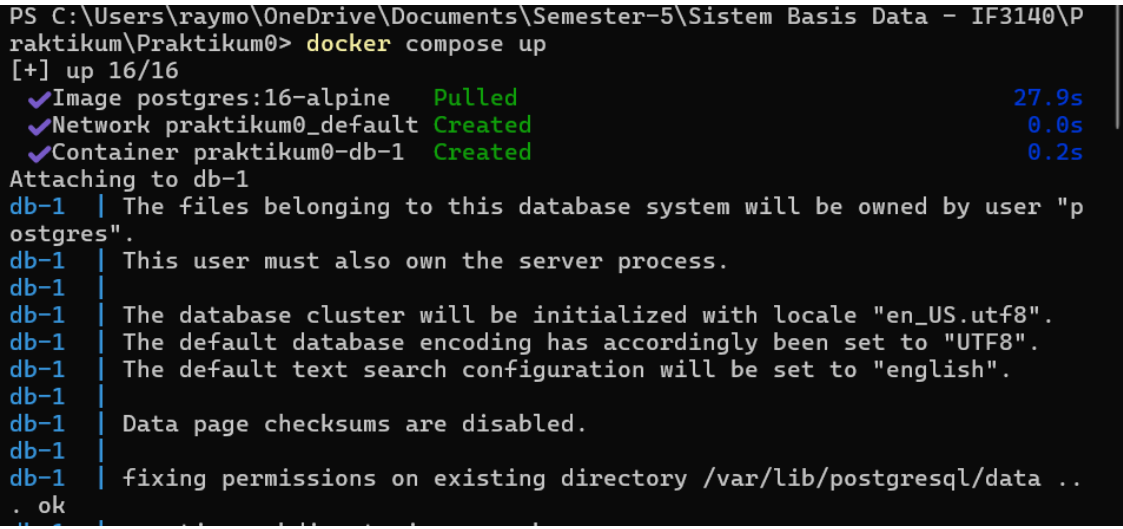
```
film = (idfilm, namafilm, tahun, durasi, imdbrating)
film_has_genre = (idfilm, genre)
film_has_kemampuanfilm = (idfilm, kemampuan)
genre = (genre)
kemampuanfilm = (kemampuan)
kursi = (idkursi, idstudio)
pelanggan = (idpelanggan, namapelanggan)
pelangganonline = (idpelanggan, email, passwordhash)
studio = (idstudio, namastudio)
studio_has_kemampuanfilm = (idstudio, kemampuan)
reservasi = (idfilm, idstudio, waktumulai, idkursi, idpelanggan)
penayangan = (idfilm, idstudio, waktumulai)
penayangan_has_kemampuanfilm = (idfilm, idstudio, waktumulai, kemampuan)
```

Silahkan eksplorasi *primary key*, *foreign key*, dan properti lainnya langsung melalui basis data.

Bagian I. Operasi Dasar PostgreSQL

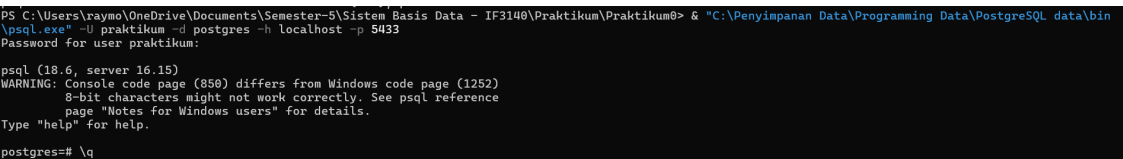
1. Nyalakan Server PostgreSQL

Jalankan file *docker* yang tersedia.

Command Terminal	<code>docker compose up</code>
SS Terminal dan Hasil	
 <pre> PS C:\Users\raymo\OneDrive\Documents\Semester-5\Sistem Basis Data - IF3140\Praktikum\Praktikum0> docker compose up [+] up 16/16 ✓Image postgres:16-alpine Pulled 27.9s ✓Network praktikum0_default Created 0.0s ✓Container praktikum0-db-1 Created 0.2s Attaching to db-1 db-1 The files belonging to this database system will be owned by user "postgres". db-1 This user must also own the server process. db-1 db-1 The database cluster will be initialized with locale "en_US.utf8". db-1 The default database encoding has accordingly been set to "UTF8". db-1 The default text search configuration will be set to "english". db-1 db-1 Data page checksums are disabled. db-1 db-1 fixing permissions on existing directory /var/lib/postgresql/data .. . ok </pre>	

2. Masuk ke PostgreSQL

Login ke PostgreSQL. Port yang digunakan adalah 5433, dengan **username praktikum** dan **password labdas**

Command Terminal	<code>psql -U praktikum -d postgres -h localhost -p 5433</code>
SS Terminal dan Hasil	
 <pre> PS C:\Users\raymo\OneDrive\Documents\Semester-5\Sistem Basis Data - IF3140\Praktikum\Praktikum0> "C:\Penyimpanan Data\Programming Data\PostgreSQL data\bin\psql.exe" -U praktikum -d postgres -h localhost -p 5433 Password for user praktikum: psql (18.6, server 16.15) WARNING: Console code page (850) differs from Windows code page (1252) 8-bit characters might not work correctly. See psql reference page "Notes for Windows users" for details. Type "help" for help. postgres=# \q </pre>	
* disini saya menggunakan path dari psql.exe karena saya menggunakan custom penempatannya	

3. Daftar Basis Data

Tampilkan daftar basis data yang ada.

Query	\l
SS Query dan Hasil	
<pre>postgres=# \l List of databases Name Owner Encoding Locale Provider Collate Ctype Locale ICU Rules Access privileges -----+-----+-----+-----+-----+-----+-----+-----+----- bioskop praktikum UTF8 libc en_US.utf8 en_US.utf8 =c/praktikum postgres praktikum UTF8 libc en_US.utf8 en_US.utf8 praktikum=C/c/praktikum template0 praktikum UTF8 libc en_US.utf8 en_US.utf8 =c/praktikum template1 praktikum UTF8 libc en_US.utf8 en_US.utf8 praktikum=C/c/praktikum (4 rows)</pre>	

4. Menuju Basis Data

Masuk ke basis data bioskop.

Query	\c bioskop
SS Query dan Hasil	
<pre>postgres=# \c bioskop psql (18.6, server 16.15) You are now connected to database "bioskop" as user "praktikum".</pre>	

5. Daftar Tabel

Tampilkan daftar tabel yang ada pada basis data bioskop.

Query	\dt
SS Query dan Hasil	
<pre>bioskop=# \dt List of tables Schema Name Type Owner -----+-----+-----+----- public _film table praktikum public _film_has_genre table praktikum public _film_has_kemampuanfilm table praktikum public _genre table praktikum public _kemampuanfilm table praktikum public _kursi table praktikum public _pelanggan table praktikum public _pelangganonline table praktikum public _penayangan table praktikum public _penayangan_has_kemampuanfilm table praktikum public _reservasi table praktikum public _studio table praktikum public _studio_has_kemampuanfilm table praktikum (13 rows)</pre>	

6. Keyword EXPLAIN

Apa itu keyword EXPLAIN?

rencana yang akan digunakan PostgreSQL untuk mengeksekusi query tersebut (query execution plan)

Berikan satu contoh penggunaan keyword EXPLAIN. Lalu, jelaskan apa yang terjadi!

Query	EXPLAIN SELECT * FROM film WHERE tahun > 2000;
SS Query dan Hasil	
<pre> biroskop=# explain biroskop=# select * biroskop=# from _film biroskop=# where tahun > 2000; QUERY PLAN ----- Seq Scan on _film (cost=0.00..2.56 rows=73 width=28) Filter: (tahun > 2000) (2 rows) </pre>	
Penjelasan	
Query menggunakan EXPLAIN untuk melihat execution plan dari pencarian film dengan tahun rilis setelah 2000. PostgreSQL memilih Sequential Scan, yaitu membaca seluruh baris pada tabel <code>_film</code>, kemudian melakukan filter terhadap kondisi <code>tahun > 2000</code>. PostgreSQL memperkirakan query menghasilkan 73 baris dengan rata-rata ukuran 28 byte per baris dan total cost sebesar 2.56.	

7. Properti \timing

Apa itu properti \timing?

Perintah pada psql yang digunakan untuk menampilkan waktu yang dibutuhkan PostgreSQL dalam mengeksekusi suatu query.

Berikan satu contoh penggunaan properti \timing. Lalu, jelaskan apa yang terjadi!

Query	\timing SELECT * FROM film WHERE tahun > 2000;			
SS Query dan Hasil				
bioskop=# \timing Timing is on.				
bioskop=# select * bioskop=# from _film bioskop=# where tahun > 2000;				
idfilm	namafilm	tahun	durasi	imdbrating
28	Jurassic World	2015	124	7.00
69	Hugo	2011	126	7.50
80	Snow White and the Huntsman	2012	132	6.10
268	Live Free or Die Hard	2007	129	7.20
303	Epic	2013	102	6.70
343	Jurassic Park III	2001	92	5.90
488	Evolution	2015	81	6.40
505	Pompeii	2014	105	5.60
562	Minions	2015	91	6.40
902	Moulin Rouge!	2001	127	7.60
926	Shrek	2001	90	7.90
999	Hannibal Rising	2007	131	6.20
1021	Fair Game	2010	108	6.80
3912	The Night Listener	2006	81	5.90
3933	Zipper	2015	103	5.70
4012	The Betrayed	2008	98	6.10
4032	The Hunt	2012	115	8.30
4087	Water	2005	117	7.80
4147	Amnesiac	2014	90	4.30
4439	Fetching Cody	2005	87	6.80
4600	Alone with Her	2006	78	6.10
4693	King Kong	2005	201	7.20
4804	The Dark Hours	2005	80	6.10
4832	Wendy and Lucy	2008	80	7.10
4942	An American in Hollywood	2014	89	7.20
4977	The FP	2011	82	5.60
(73 rows)				
Time: 3.887 ms				

Penjelasan
Setelah \timing aktif, query dijalankan seperti biasa, tetapi PostgreSQL juga menampilkan waktu yang dibutuhkan untuk menyelesaikan query tersebut. Nilai waktu ditampilkan dalam satuan milidetik pada bagian akhir hasil query.

Bagian II. PostgreSQL DDL & DML

Kerjakan No 1-5 terlebih dahulu. Selanjutnya, kerjakan No 6-11 secara berurutan. Untuk seluruh soal, aktifkan `\timing` dan masukkan hasil perhitungan waktu dalam tangkapan layar.

1. Rekap Penjualan Tiket

Tampilkan berapa jumlah kursi yang dipeservasi per bulan, urutkan dari bulan yang memiliki reservasi kursi paling banyak.

Query	<pre>SELECT TO_CHAR(waktumulai::timestamp, 'YYYY-MM') AS bulan, COUNT(*) AS jumlah_kursi FROM _reservasi GROUP BY TO_CHAR(waktumulai::timestamp, 'YYYY-MM') ORDER BY jumlah_kursi DESC;</pre>
SS Query dan Hasil	

```

bioskop=# SELECT
bioskop=#     TO_CHAR(waktumulai::timestamp, 'YYYY-MM') AS bulan,
bioskop=#     COUNT(*) AS jumlah_kursi
bioskop=# FROM _reservasi
bioskop=# GROUP BY TO_CHAR(waktumulai::timestamp, 'YYYY-MM')
bioskop=# ORDER BY jumlah_kursi DESC;

```

bulan	jumlah_kursi
2008-05	2593
2014-01	2583
1900-03	1960
1995-05	1857
2006-01	1813
2001-09	1804
2012-08	1764
1995-04	1717
1995-12	1543
2002-11	1532
1998-08	1416
2011-01	1370
2005-09	1366
1900-12	1333
1993-12	1300
1900-08	1224
1997-06	1188
2004-11	1168
1999-12	1167
2011-12	1153
1998-03	1139
2005-08	1136
2006-06	1135
2015-08	1124
1998-11	1121
1929-07	1116
2005-07	1104
2002-07	1086
1900-01	1080
1997-01	1051
1900-02	1033
2013-11	1030
1900-04	1010

```

2003-10 | 556
2007-10 | 545
2006-02 | 543
1981-03 | 537
2004-08 | 518
2015-12 | 515
2000-04 | 501
2003-03 | 498
1994-12 | 492
2011-05 | 489
1994-01 | 477
2002-08 | 469
1980-07 | 455
2016-06 | 444
1995-03 | 406
2012-06 | 394
1999-07 | 391
2014-10 | 385
2012-09 | 368
1994-10 | 357
2003-12 | 356
1993-03 | 310
2012-07 | 292
2015-10 | 283
1981-02 | 274
2014-06 | 265
1999-08 | 234
1916-05 | 227
2008-12 | 221
1999-03 | 202
2016-07 | 185
2001-05 | 144
2015-07 | 128
2008-08 | 121
1998-04 | 113
2008-06 | 111
1995-02 | 99
1989-10 | 87
2013-12 | 66
2010-11 | 65
1997-10 | 36
(130 rows)

```

Time: 129.857 ms

2. Genre Film Terfavorit

Tampilkan genre terfavorit (ditunjukkan dengan penjualan tiket yang paling tinggi) dari seluruh film yang dirilis setelah tahun 1995. Berikan pula total jumlah tiket yang terjual

Query	SQL
	<pre> SELECT fg.genre, COUNT(r.idkursi) AS total_terjual FROM _reservasi r JOIN _film_has_genre fg </pre>

	<pre> ON r.idfilm = fg.idfilm JOIN _film f ON r.idfilm = f.idfilm WHERE f.tahun > 1995 GROUP BY fg.genre ORDER BY total_terjual DESC; </pre>
SS Query dan Hasil	
<pre> bioskop=# SELECT bioskop-# fg.genre, bioskop-# COUNT(r.idkursi) AS total_terjual bioskop-# FROM _reservasi r bioskop-# JOIN _film_has_genre fg bioskop-# ON r.idfilm = fg.idfilm bioskop-# JOIN _film f bioskop-# ON r.idfilm = f.idfilm bioskop-# WHERE f.tahun > 1995 bioskop-# GROUP BY fg.genre bioskop-# ORDER BY total_terjual DESC; genre total_terjual -----+----- Drama 44744 Thriller 23404 Crime 19858 Comedy 18136 Romance 16375 Adventure 13390 Action 11444 Mystery 9483 Family 8084 Fantasy 6759 Sci-Fi 6619 Biography 5797 Horror 5225 Animation 2874 Sport 2726 Musical 2446 Music 1686 War 1682 History 1636 (19 rows) Time: 63.728 ms </pre>	

3. Studio Elite

Studio Elite didefinisikan sebagai studio yang memiliki kemampuan 3D, dan memiliki jumlah kursi lebih dari 150. Tampilkan id, nama, dan jumlah kursi dari seluruh Studio Elite yang ada.

Query	<pre> SELECT s.idstudio AS id, </pre>
-------	--

```

        st.namastudio AS nama,

        COUNT(k.idkursi) AS jumlah_kursi

FROM _studio_has_kemampuanfilm s

JOIN _kursi k

        ON s.idstudio = k.idstudio

JOIN _studio st

        ON s.idstudio = st.idstudio

WHERE s.kemampuan = '3D'

GROUP BY s.idstudio, st.namastudio

HAVING COUNT(k.idkursi) > 150;

```

SS Query dan Hasil

```

Time: 19.504 ms
bioskop=# SELECT
bioskop=#      s.idstudio AS id,
bioskop=#      st.namastudio AS nama,
bioskop=#      COUNT(k.idkursi) AS jumlah_kursi
bioskop=# FROM _studio_has_kemampuanfilm s
bioskop=# JOIN _kursi k
bioskop=#      ON s.idstudio = k.idstudio
bioskop=# JOIN _studio st
bioskop=#      ON s.idstudio = st.idstudio
bioskop=# WHERE s.kemampuan = '3D'
bioskop=# GROUP BY s.idstudio, st.namastudio
bioskop=# HAVING COUNT(k.idkursi) > 150;
 id |  nama  | jumlah_kursi
-----+-----+-----
  6 | Studio 7 |         180
 17 | Studio 18 |         220
 16 | Studio 17 |         275
 10 | Studio 11 |         180
 14 | Studio 15 |         200
  2 | Studio 3  |         180
 13 | Studio 14 |         180
(7 rows)

```

Time: 19.504 ms

4. Studio Jauh

Tampilkan id film yang ditayangkan di studio 10 hingga studio 19 pada bulan Januari hingga September tahun 2015 dan memiliki rating lebih dari 6.

Query	<pre>SELECT DISTINCT f.idfilm FROM _penayangan p JOIN _film f ON p.idfilm = f.idfilm WHERE p.idstudio BETWEEN 10 AND 19 AND TO_CHAR(p.waktumulai::timestamp, 'YYYY-MM') LIKE '2015-0%' AND f.imdbrating > 6;</pre>
SS Query dan Hasil	
<pre>biroskop=# SELECT DISTINCT f.idfilm biroskop=# FROM _penayangan p biroskop=# JOIN _film f biroskop=# ON p.idfilm = f.idfilm biroskop=# WHERE p.idstudio BETWEEN 10 AND 19 biroskop=# AND TO_CHAR(p.waktumulai::timestamp, 'YYYY-MM') LIKE '2015-0%' biroskop=# AND f.imdbrating > 6; idfilm ----- 28 488 562 (3 rows) Time: 10.507 ms</pre>	

5. Komunitas Marcus

Buatlah sebuah view bernama “marcus_movie” yang menampilkan judul film sebagai “Judul”, rating IMDB sebagai “Rating Film”, dan tahun rilis sebagai “Tahun” dari film-film yang pernah dilihat oleh pelanggan yang memiliki nama depan Marcus! Tunjukkan juga bahwa view berhasil dibuat!

Query Jawaban	<pre>CREATE VIEW marcus_movie AS SELECT DISTINCT</pre>
---------------	--

	<pre> f.namafilm AS "Judul", f.imdbrating AS "Rating Film", f.tahun AS "Tahun" FROM _pelanggan p JOIN _reservasi r ON p.idpelanggan = r.idpelanggan JOIN _film f ON r.idfilm = f.idfilm WHERE SPLIT_PART(p.namapelanggan, ' ', 1) = 'Marcus'; </pre>
Query Testing	<pre> SELECT * FROM marcus_movie; </pre>
SS Query Sebelum Manipulasi Menggunakan Query Testing	
<pre> bioskop=# SELECT * FROM marcus_movie; ERROR: relation "marcus_movie" does not exist LINE 1: SELECT * FROM marcus_movie; ^ Time: 7.732 ms </pre>	
SS Query Saat Manipulasi Menggunakan Query Jawaban	
<pre> bioskop=# CREATE VIEW marcus_movie AS bioskop=# SELECT DISTINCT bioskop=# f.namafilm AS "Judul", bioskop=# f.imdbrating AS "Rating Film", bioskop=# f.tahun AS "Tahun" bioskop=# FROM _pelanggan p bioskop=# JOIN _reservasi r bioskop=# ON p.idpelanggan = r.idpelanggan bioskop=# JOIN _film f bioskop=# ON r.idfilm = f.idfilm bioskop=# WHERE SPLIT_PART(p.namapelanggan, ' ', 1) = 'Marcus'; CREATE VIEW Time: 72.666 ms </pre>	
SS Query Setelah Manipulasi Menggunakan Query Testing	

```

bioskop=# SELECT * FROM marcus_movie;
          Judul          | Rating Film | Tahun
-----+-----+-----
 40 Days and 40 Nights   |         5.60 | 2002
 Formula 51              |         6.30 | 2001
 Niagara                 |         7.10 | 1953
 Scream: The TV Series   |         7.30 | 1900
 Swing Vote              |         6.10 | 2008
(5 rows)

Time: 71.424 ms

```

6. Populer di Genrenya

Satu film dapat memiliki lebih dari satu genre. Buatlah View dengan nama 'above_avg_genre_penayangan' yang menampilkan genre sebagai 'nama_genre' dan jumlah penayangannya sebagai 'jumlah_penayangan' untuk genre film yang memiliki jumlah penayangan di atas rata-rata. Tampilkan hasil akhir viewnya!

Query Jawaban	<pre> CREATE OR REPLACE VIEW above_avg_genre_penayangan AS WITH jumlah_per_genre AS (SELECT f.genre AS nama_genre, COUNT(p.idfilm) AS jumlah_penayangan FROM _penayangan p JOIN _film_has_genre f ON p.idfilm = f.idfilm GROUP BY f.genre) SELECT nama_genre, jumlah_penayangan FROM jumlah_per_genre WHERE jumlah_penayangan > (SELECT AVG(jumlah_penayangan) FROM jumlah_per_genre </pre>
---------------	---

	);
Query Testing	Select * from above_avg_genre_penayangan;
SS Query Sebelum Manipulasi Menggunakan Query Testing	
<pre> bioskop=# Select * from above_avg_genre_penayangan bioskop-# ; ERROR: relation "above_avg_genre_penayangan" does not exist LINE 1: Select * from above_avg_genre_penayangan ^ Time: 12.116 ms </pre>	
SS Query Saat Manipulasi Menggunakan Query Jawaban	
<pre> bioskop=# CREATE OR REPLACE VIEW above_avg_genre_penayangan AS bioskop-# WITH jumlah_per_genre AS (bioskop(# SELECT bioskop(# f.genre AS nama_genre, bioskop(# COUNT(p.idfilm) AS jumlah_penayangan bioskop(# FROM _penayangan p bioskop(# JOIN _film_has_genre f bioskop(# ON p.idfilm = f.idfilm bioskop(# GROUP BY f.genre bioskop(#) bioskop-# SELECT bioskop-# nama_genre, bioskop-# jumlah_penayangan bioskop-# FROM jumlah_per_genre bioskop-# WHERE jumlah_penayangan > (bioskop(# SELECT AVG(jumlah_penayangan) bioskop(# FROM jumlah_per_genre bioskop(#); CREATE VIEW Time: 22.652 ms </pre>	
SS Query Setelah Manipulasi Menggunakan Query Testing	
<pre> bioskop=# Select * from above_avg_genre_penayangan; nama_genre jumlah_penayangan -----+----- Crime 286 Romance 191 Mystery 136 Drama 592 Action 171 Thriller 294 Comedy 240 Adventure 162 (8 rows) Time: 13.598 ms </pre>	

7. Tahun Baru Bersama Pacar

Veronica Doheny selalu membuat sebuah reservasi dengan pacarnya dengan menonton film yang tayang di tahun baru. Tetapi, mereka tidak akan menonton film yang memiliki rating dibawah 7. Namun, pada pada malam tahun baru, selalu saja terdapat bug sehingga transaksi yang ia lakukan tidak tercatat. Bantu pegawai untuk menambahkan pemesanan tiket pada tahun baru 2011. (Kerjakan hanya dengan 1 query saja, untuk letak kursi dibebaskan asalkan bersebelahan dengan asumsi seluruh kursi masih kosong)

Query Jawaban	<pre> WITH target_penayangan AS (SELECT p.idfilm, p.idstudio, p.waktumulai FROM _penayangan p JOIN _film f ON p.idfilm = f.idfilm WHERE f.imdbrating >= 7 AND p.waktumulai::timestamp::date = DATE '2011-01-01') INSERT INTO _reservasi (idfilm, idstudio, waktumulai, idkursi, idpelanggan) SELECT t.idfilm, t.idstudio, t.waktumulai, kursi.idkursi, 15932 FROM target_penayangan t CROSS JOIN (VALUES ('A1'), ('A2')) AS kursi(idkursi); </pre>
Query Testing	<pre> SELECT p.namapelanggan, r.idfilm, </pre>

```

        f.namafilm,
        f.imdbrating,
        r.idstudio,
        r.waktumulai,
        r.idkursi
FROM _reservasi r
JOIN _pelanggan p
    ON r.idpelanggan = p.idpelanggan
JOIN _film f
    ON r.idfilm = f.idfilm
WHERE p.namapelanggan = 'Veronica Doheny'
    AND r.waktumulai::timestamp::date = DATE '2011-01-01'
ORDER BY r.idkursi;

```

SS Query Sebelum Manipulasi Menggunakan Query Testing

```

bioskop=# SELECT
bioskop=#     p.namapelanggan,
bioskop=#     r.idfilm,
bioskop=#     f.namafilm,
bioskop=#     f.imdbrating,
bioskop=#     r.idstudio,
bioskop=#     r.waktumulai,
bioskop=#     r.idkursi
bioskop=# FROM _reservasi r
bioskop=# JOIN _pelanggan p
bioskop=#     ON r.idpelanggan = p.idpelanggan
bioskop=# JOIN _film f
bioskop=#     ON r.idfilm = f.idfilm
bioskop=# WHERE p.namapelanggan = 'Veronica Doheny'
bioskop=#     AND r.waktumulai::timestamp::date = DATE '2011-01-01'
bioskop=# ORDER BY r.idkursi;
  namapelanggan | idfilm | namafilm | imdbrating | idstudio | waktumulai | idkursi
-----+-----+-----+-----+-----+-----+-----
(0 rows)

Time: 126.832 ms

```

SS Query Saat Manipulasi Menggunakan Query Jawaban

```

bioskop=# WITH target_penayangan AS (
bioskop(# SELECT
bioskop(#     p.idfilm,
bioskop(#     p.idstudio,
bioskop(#     p.waktumulai
bioskop(# FROM _penayangan p
bioskop(# JOIN _film f
bioskop(#     ON p.idfilm = f.idfilm
bioskop(# WHERE f.imdbrating >= 7
bioskop(#     AND p.waktumulai::timestamp::date = DATE '2011-01-01'
bioskop(# )
bioskop=# INSERT INTO _reservasi
bioskop=#     (idfilm, idstudio, waktumulai, idkursi, idpelanggan)
bioskop=# SELECT
bioskop=#     t.idfilm,
bioskop=#     t.idstudio,
bioskop=#     t.waktumulai,
bioskop=#     kursi.idkursi,
bioskop=#     15932
bioskop=# FROM target_penayangan t
bioskop=# CROSS JOIN (
bioskop(#     VALUES ('A1'), ('A2')
bioskop(# ) AS kursi(idkursi);
INSERT 0 2
Time: 21.783 ms

```

SS Query Setelah Manipulasi Menggunakan Query Testing

```

bioskop=# SELECT
bioskop=#     p.namapelanggan,
bioskop=#     r.idfilm,
bioskop=#     f.namafilm,
bioskop=#     f.imdbrating,
bioskop=#     r.idstudio,
bioskop=#     r.waktumulai,
bioskop=#     r.idkursi
bioskop=# FROM _reservasi r
bioskop=# JOIN _pelanggan p
bioskop=#     ON r.idpelanggan = p.idpelanggan
bioskop=# JOIN _film f
bioskop=#     ON r.idfilm = f.idfilm
bioskop=# WHERE p.namapelanggan = 'Veronica Doheny'
bioskop=#     AND r.waktumulai::timestamp::date = DATE '2011-01-01'
bioskop=# ORDER BY r.idkursi;

```

namapelanggan	idfilm	namafilm	imdbrating	idstudio	waktumulai	idkursi
Veronica Doheny	2420	Soul Surfer	7.10	3	2011-01-01 17:00:00	A1
Veronica Doheny	2420	Soul Surfer	7.10	3	2011-01-01 17:00:00	A2

(2 rows)

```

Time: 54.186 ms

```

8. Rating Masking

Pihak bioskop ingin memperbarui rating beberapa film berdasarkan jumlah penontonnya. Buatlah query untuk mengupdate rating film yang memiliki jumlah

penonton diatas 500 dan rating lebih kecil dari 6.00 menjadi 6.00. Tunjukkan bahwa tidak ada lagi film yang memiliki penonton diatas 500 yang memiliki rating dibawah 6.00.

Query Jawaban	<pre> UPDATE _film f SET imdbrating = 6.00 WHERE f.imdbrating < 6.00 AND f.idfilm IN (SELECT r.idfilm FROM _reservasi r GROUP BY r.idfilm HAVING COUNT(r.idkursi) > 500); </pre>
Query Testing	<pre> SELECT f.idfilm, f.namafilm, f.imdbrating, COUNT(r.idkursi) AS jumlah_penonton FROM _film f JOIN _reservasi r ON f.idfilm = r.idfilm GROUP BY f.idfilm, f.namafilm, f.imdbrating HAVING COUNT(r.idkursi) > 500 AND f.imdbrating < 6.00; </pre>
SS Query Sebelum Manipulasi Menggunakan Query Testing	

```

bioskop=# SELECT
bioskop-#     f.idfilm,
bioskop-#     f.namafilam,
bioskop-#     f.imdbrating,
bioskop-#     COUNT(r.idkursi) AS jumlah_penonton
bioskop-# FROM _film f
bioskop-# JOIN _reservasi r
bioskop-#     ON f.idfilm = r.idfilm
bioskop-# GROUP BY
bioskop-#     f.idfilm,
bioskop-#     f.namafilam,
bioskop-#     f.imdbrating
bioskop-# HAVING COUNT(r.idkursi) > 500
bioskop-#     AND f.imdbrating < 6.00;

```

idfilm	namafilam	imdbrating	jumlah_penonton
3218	The Brown Bunny	5.00	556
4977	The FP	5.60	665
3912	The Night Listener	5.90	631
1788	Pride and Prejudice and Zombies	5.80	662
505	Pompeii	5.60	559
3394	Woman on Top	5.30	501
3294	Breakin' All the Rules	5.50	609
564	Snake Eyes	5.90	692
499	Babe: Pig in the City	5.80	661
1690	The Whole Ten Yards	5.50	819
1334	Against the Ropes	5.30	518
758	Under Siege 2: Dark Territory	5.40	505
2019	Point Break	5.30	515
3945	Among Giants	5.90	597
3933	Zipper	5.70	618
2855	No Good Deed	5.60	582
2497	40 Days and 40 Nights	5.60	621
343	Jurassic Park III	5.90	694
1209	Ride Along 2	5.90	629

(19 rows)

Time: 71.880 ms

SS Query Saat Manipulasi Menggunakan Query Jawaban

```

bioskop=# UPDATE _film f
bioskop-# SET imdbrating = 6.00
bioskop-# WHERE f.imdbrating < 6.00
bioskop-#     AND f.idfilm IN (
bioskop(#     SELECT r.idfilm
bioskop(#     FROM _reservasi r
bioskop(#     GROUP BY r.idfilm
bioskop(#     HAVING COUNT(r.idkursi) > 500
bioskop(# );
UPDATE 19
Time: 32.916 ms

```

SS Query Setelah Manipulasi Menggunakan Query Testing

```

bioskop=# UPDATE _film f
bioskop=# SET imdbrating = 6.00
bioskop=# WHERE f.imdbrating < 6.00
bioskop=# AND f.idfilm IN (
bioskop(# SELECT r.idfilm
bioskop(# FROM _reservasi r
bioskop(# GROUP BY r.idfilm
bioskop(# HAVING COUNT(r.idkursi) > 500
bioskop(# );
UPDATE 19
Time: 32.916 ms
bioskop=#
bioskop=# SELECT
bioskop=# f.idfilm,
bioskop=# f.namafilm,
bioskop=# f.imdbrating,
bioskop=# COUNT(r.idkursi) AS jumlah_penonton
bioskop=# FROM _film f
bioskop=# JOIN _reservasi r
bioskop=# ON f.idfilm = r.idfilm
bioskop=# GROUP BY
bioskop=# f.idfilm,
bioskop=# f.namafilm,
bioskop=# f.imdbrating
bioskop=# HAVING COUNT(r.idkursi) > 500
bioskop=# AND f.imdbrating < 6.00;
 idfilm | namafilm | imdbrating | jumlah_penonton
-----+-----+-----+-----
(0 rows)

Time: 12.496 ms

```

9. SPG Studio

Buatlah sebuah tabel dengan nama SPG yang berisi id karyawan, id studio, id film yang sedang ditayangkan. Perhatikan kolom yang harus direferensikan ke table lain.

Query	ALTER TABLE _film
-------	-------------------

Jawaban	<pre> ADD CONSTRAINT uq_film_idfilm UNIQUE (idfilm); ALTER TABLE _studio ADD CONSTRAINT uq_studio_idstudio UNIQUE (idstudio); CREATE TABLE SPG (idkaryawan INTEGER PRIMARY KEY, idstudio SMALLINT, idfilm SMALLINT, CONSTRAINT fk_spg_studio FOREIGN KEY (idstudio) REFERENCES _studio(idstudio), CONSTRAINT fk_spg_film FOREIGN KEY (idfilm) REFERENCES _film(idfilm)); </pre>
Query Testing	<pre> SELECT table_name FROM information_schema.tables WHERE table_schema = 'public' AND LOWER(table_name) = 'spg'; </pre>
SS Query Sebelum Manipulasi Menggunakan Query Testing	
<pre> bioskop=# SELECT table_name bioskop=# FROM information_schema.tables bioskop=# WHERE table_schema = 'public' bioskop=# AND LOWER(table_name) = 'spg'; table_name ----- (0 rows) </pre>	
SS Query Saat Manipulasi Menggunakan Query Jawaban	

```

bioskop=# ALTER TABLE _film
bioskop=# ADD CONSTRAINT uq_film_idfilm UNIQUE (idfilm);
ALTER TABLE
Time: 24.525 ms
bioskop=#
bioskop=# ALTER TABLE _studio
bioskop=# ADD CONSTRAINT uq_studio_idstudio UNIQUE (idstudio);
ALTER TABLE
Time: 5.983 ms
bioskop=#
bioskop=# CREATE TABLE SPG (
bioskop(#      idkaryawan INTEGER PRIMARY KEY,
bioskop(#      idstudio SMALLINT,
bioskop(#      idfilm SMALLINT,
bioskop(#
bioskop(#      CONSTRAINT fk_spg_studio
bioskop(#      FOREIGN KEY (idstudio)
bioskop(#      REFERENCES _studio(idstudio),
bioskop(#
bioskop(#      CONSTRAINT fk_spg_film
bioskop(#      FOREIGN KEY (idfilm)
bioskop(#      REFERENCES _film(idfilm)
bioskop(# );
CREATE TABLE
Time: 35.599 ms

```

SS Query Setelah Manipulasi Menggunakan Query Testing

```

bioskop=# SELECT table_name
bioskop=# FROM information_schema.tables
bioskop=# WHERE table_schema = 'public'
bioskop=# AND LOWER(table_name) = 'spg';
table_name
-----
spg
(1 row)

Time: 11.513 ms

```

10. Tally Counter

Setiap kali ada pelanggan melakukan reservasi, maka jumlah penonton film tersebut harus otomatis bertambah. Untuk itu:

1. Tambahkan kolom baru jumlahpenonton pada tabel film dengan nilai awal 0.
2. Buat trigger pada tabel reservasi yang akan menambahkan +1 pada kolom jumlah penonton di tabel film sesuai idfilm yang dipesan.

Query Case	<pre>SELECT idfilm, namafilm, tahun, durasi, imdbrating, jumlahpenonton FROM _film ORDER BY jumlahpenonton DESC;</pre>
SS Query Case dan Hasil	
<pre> bioskop=# ALTER TABLE _film bioskop=# ADD COLUMN jumlahpenonton INTEGER NOT NULL DEFAULT 0; ALTER TABLE Time: 15.834 ms bioskop=# SELECT idfilm, namafilm, tahun, durasi, imdbrating, jumlahpenonton bioskop=# FROM _film bioskop=# ORDER BY jumlahpenonton DESC; idfilm namafilm tahun durasi imdbrating jumlahpenonton -----+-----+-----+-----+-----+----- 28 Jurassic World 2015 124 7.00 0 69 Hugo 2011 126 7.50 0 80 Snow White and the Huntsman 2012 132 6.10 0 216 Batman & Robin 1997 125 3.70 0 268 Live Free or Die Hard 2007 129 7.20 0 303 Epic 2013 102 6.70 0 488 Evolution 2015 81 6.40 0 562 Minions 2015 91 6.40 0 702 The American President 1995 114 6.80 0 902 Moulin Rouge! 2001 127 7.60 0 926 Shrek 2001 90 7.90 0 961 Unforgotten 1900 45 7.90 0 987 On Deadly Ground 1994 101 4.40 0 999 Hannibal Rising 2007 131 6.20 0 1021 Fair Game 2010 108 6.80 0 1111 Dune 1984 177 6.60 0 1161 High Crimes 2002 115 6.30 0 1190 Spy Kids 3-D: Game Over 2003 84 4.10 0 1237 Predators 2010 107 6.40 0 1244 Jersey Boys 2014 134 6.90 0 1327 The Crow 1994 98 7.60 0 1416 Sky High 2005 100 6.20 0 1448 From Hell 2001 122 6.80 0 1455 Predator 2 1990 103 6.20 0 1476 The Four Feathers 2002 125 6.50 0 0 (125 rows) Time: 5.563 ms </pre>	
Query Jawaban	<pre>ALTER TABLE _film ADD COLUMN jumlahpenonton INTEGER NOT NULL DEFAULT 0; CREATE OR REPLACE FUNCTION tambah_jumlah_penonton() RETURNS TRIGGER AS \$\$ BEGIN</pre>

	<pre> UPDATE _film SET jumlahpenonton = jumlahpenonton + 1 WHERE idfilm = NEW.idfilm; RETURN NEW; END; \$\$ LANGUAGE plpgsql; CREATE TRIGGER trigger_tambah_jumlah_penonton AFTER INSERT ON _reservasi FOR EACH ROW EXECUTE FUNCTION tambah_jumlah_penonton(); </pre>
SS Query Jawaban dan Hasil	
	<pre> bioskop=# bioskop=# CREATE OR REPLACE FUNCTION tambah_jumlah_penonton() bioskop-# RETURNS TRIGGER AS \$\$ bioskop\$# BEGIN bioskop\$# UPDATE _film bioskop\$# SET jumlahpenonton = jumlahpenonton + 1 bioskop\$# WHERE idfilm = NEW.idfilm; bioskop\$# bioskop\$# RETURN NEW; bioskop\$# END; bioskop\$# \$\$ LANGUAGE plpgsql; CREATE FUNCTION Time: 44.243 ms bioskop=# bioskop=# CREATE TRIGGER trigger_tambah_jumlah_penonton bioskop-# AFTER INSERT ON _reservasi bioskop-# FOR EACH ROW bioskop-# EXECUTE FUNCTION tambah_jumlah_penonton(); CREATE TRIGGER Time: 8.446 ms </pre>
Query Testing	<pre> INSERT INTO _reservasi (idfilm, idstudio, waktumulai, idkursi, idpelanggan) VALUES (28, 0, '2025-10-01 19:00:00', 'A1', 0); </pre>
SS Query Case dan Hasil Setelah Query Testing	

```

bioskop=# INSERT INTO _reservasi
bioskop=#      (idfilm, idstudio, waktumulai, idkursi, idpelanggan)
bioskop=# VALUES
bioskop=#      (28, 0, '2025-10-01 19:00:00', 'A1', 0);
INSERT 0 1
Time: 16.083 ms

```

```

bioskop=# SELECT idfilm, namafilm, tahun, durasi, imdbrating, jumlahpenonton
bioskop=# FROM _film
bioskop=# ORDER BY jumlahpenonton DESC;

```

idfilm	namafilm	tahun	durasi	imdbrating	jumlahpenonton
28	Jurassic World	2015	124	7.00	1
80	Snow White and the Huntsman	2012	132	6.10	0
216	Batman & Robin	1997	125	3.70	0
268	Live Free or Die Hard	2007	129	7.20	0
303	Enic	2013	102	6.70	0

11. Autofill

Buatlah trigger agar setiap kali ada data baru masuk ke tabel reservasi tanpa diisi kolom waktumulai, sistem akan secara otomatis mengisi kolom waktumulai sesuai jadwal dari tabel penayangan (berdasarkan idfilm dan idstudio).

Query Case	SELECT * FROM _reservasi;				
SS Query Case dan Hasil					
bioskop=# SELECT * FROM _reservasi;					
idfilm	idstudio	waktumulai	idkursi	idpelanggan	
28	0	2015-09-30 09:00:00	A10	14110	
28	0	2015-09-30 09:00:00	A11	14110	
28	0	2015-09-30 09:00:00	A12	14110	
28	0	2015-09-30 09:00:00	A13	14110	
28	0	2015-09-30 09:00:00	A7	10717	
28	0	2015-09-30 09:00:00	A8	10717	
28	0	2015-09-30 09:00:00	A9	14110	
28	0	2015-09-30 09:00:00	B1	17509	
28	0	2015-09-30 09:00:00	B10	10113	
28	0	2015-09-30 09:00:00	B11	10113	
28	0	2015-09-30 09:00:00	B12	10113	
28	0	2015-09-30 09:00:00	B2	17509	
28	0	2015-09-30 09:00:00	B3	10113	
28	0	2015-09-30 09:00:00	B4	10113	
28	0	2015-09-30 09:00:00	B5	10113	
28	0	2015-09-30 09:00:00	B6	10113	
28	0	2015-09-30 09:00:00	B7	10113	
28	0	2015-09-30 09:00:00	B8	10113	
28	0	2015-09-30 09:00:00	B9	10113	
28	0	2015-09-30 09:00:00	C1	17509	
28	0	2015-09-30 09:00:00	C14	4258	
28	0	2015-09-30 09:00:00	C15	4258	
28	0	2015-09-30 09:00:00	C2	5975	
28	0	2015-09-30 09:00:00	C3	5975	

```

28      12      2015-09-27 09:00:00      F15      10458
28      12      2015-09-27 09:00:00      F2       15866
28      12      2015-09-27 09:00:00      F3       15866
28      12      2015-09-27 09:00:00      F4       15866
28      12      2015-09-27 09:00:00      F5       15866
28      12      2015-09-27 09:00:00      F6       15866
28      12      2015-09-27 09:00:00      F7       15866
28      12      2015-09-27 09:00:00      F8       11864
28      12      2015-09-27 09:00:00      F9       11864
28      12      2015-09-27 09:00:00      G10      13923
28      12      2015-09-27 09:00:00      G11      13923
28      12      2015-09-27 09:00:00      G12      11314
^CTime: 122.017 ms

```

Query
Jawaban

```

CREATE OR REPLACE FUNCTION autofill_waktumulai()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.waktumulai IS NULL THEN
        SELECT p.waktumulai
        INTO NEW.waktumulai
        FROM _penayangan p
        WHERE p.idfilm = NEW.idfilm
        AND p.idstudio = NEW.idstudio
        LIMIT 1;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

SS Query Jawaban dan Hasil

```

bioskop=# CREATE OR REPLACE FUNCTION autofill_waktumulai()
bioskop=# RETURNS TRIGGER AS $$
bioskop$# BEGIN
bioskop$#     IF NEW.waktumulai IS NULL THEN
bioskop$#         SELECT p.waktumulai
bioskop$#         INTO NEW.waktumulai
bioskop$#         FROM _penayangan p
bioskop$#         WHERE p.idfilm = NEW.idfilm
bioskop$#         AND p.idstudio = NEW.idstudio
bioskop$#         LIMIT 1;
bioskop$#     END IF;
bioskop$#     RETURN NEW;
bioskop$# END;
bioskop$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
Time: 7.887 ms

```

Query Testing

```

INSERT INTO _reservasi (idfilm, idstudio, idkursi,
idpelanggan)
VALUES (1, 101, 5, 2001);

SELECT * FROM reservasi
WHERE idfilm=1 AND idstudio=101 AND idkursi=5 AND
idpelanggan=2001;

(id dapat diubah sesuai kebutuhan)

```

SS Query Case dan Hasil Setelah Query Testing

```

bioskop=# INSERT INTO _reservasi (idfilm, idstudio, idkursi, idpelanggan)
bioskop=# VALUES (1, 101, 5, 2001);
INSERT 0 1
Time: 12.152 ms
bioskop=#
bioskop=# SELECT * FROM reservasi
bioskop=# WHERE idfilm=1 AND idstudio=101 AND idkursi=5 AND idpelanggan=2001;
ERROR: relation "reservasi" does not exist
LINE 1: SELECT * FROM reservasi
                        ^
Time: 6.896 ms

```

```

bioskop=# SELECT * FROM _reservasi;
idfilm | idstudio | waktumulai | idkursi | idpelanggan
-----+-----+-----+-----+-----
28 | 0 | 2015-09-30 09:00:00 | A10 | 14110
28 | 0 | 2015-09-30 09:00:00 | A11 | 14110
28 | 0 | 2015-09-30 09:00:00 | A12 | 14110
28 | 0 | 2015-09-30 09:00:00 | A13 | 14110
28 | 0 | 2015-09-30 09:00:00 | A7 | 10717
28 | 0 | 2015-09-30 09:00:00 | A8 | 10717
28 | 0 | 2015-09-30 09:00:00 | A9 | 14110
28 | 0 | 2015-09-30 09:00:00 | B1 | 17509
28 | 0 | 2015-09-30 09:00:00 | B10 | 10113
28 | 0 | 2015-09-30 09:00:00 | B11 | 10113
28 | 0 | 2015-09-30 09:00:00 | B12 | 10113
28 | 0 | 2015-09-30 09:00:00 | B2 | 17509
28 | 0 | 2015-09-30 09:00:00 | B3 | 10113
28 | 0 | 2015-09-30 09:00:00 | B4 | 10113
28 | 0 | 2015-09-30 09:00:00 | B5 | 10113
28 | 0 | 2015-09-30 09:00:00 | B6 | 10113
28 | 0 | 2015-09-30 09:00:00 | B7 | 10113
28 | 0 | 2015-09-30 09:00:00 | B8 | 10113
28 | 0 | 2015-09-30 09:00:00 | B9 | 10113
28 | 0 | 2015-09-30 09:00:00 | C1 | 17509
28 | 0 | 2015-09-30 09:00:00 | C14 | 4258
28 | 0 | 2015-09-30 09:00:00 | C15 | 4258
28 | 0 | 2015-09-30 09:00:00 | C2 | 5975
28 | 0 | 2015-09-30 09:00:00 | C3 | 5975
28 | 0 | 2015-09-30 09:00:00 | C4 | 5975
28 | 0 | 2015-09-30 09:00:00 | E6 | 5495
28 | 0 | 2015-09-30 09:00:00 | E7 | 5495
28 | 0 | 2015-09-30 09:00:00 | E8 | 9847
28 | 0 | 2015-09-30 09:00:00 | F1 | 7250
28 | 0 | 2015-09-30 09:00:00 | F10 | 11159
^CCancel request sent
Time: 98.484 ms

```